POLSKO-JAPOŃSKA
AKADEMIA TECHNIK
KOMPUTEROWYCH

ISS — LAB 1: Odometry on the AlphaBot robot

Implementation guide.

Robot: Waveshare AlphaBot with an Arduino UNO. Slotted wheel encoders.

Acceptance criterion

One criterion, taken straight from the assignment:

> 1. The operator drives the robot manually from a marked point **A** to a point **B**. 2. The robot is carried back to point **A**. A sequence of commands is sent to it over the serial interface, and the robot drives **from A to B autonomously**.

Everything else in this guide serves that single drive. The numeric values given in stages 2–4 are **diagnostic aids**, not pass/fail thresholds — they tell you whether a given building block works before you build the next one on top of it.

Functional requirements that follow from the assignment:

Requirement	Where it is covered
Driving straight for a given distance, forwards and backwards	Stage 3
Turning left/right by a given angle, resolution of at least 45°	Stage 4
Position derived **from the encoders** on the drive motors	Stage 2
Control in a **closed feedback loop** that improves accuracy	Stages 3 and 4
Your own communication protocol over the serial interface	Stage 5
Sending a command sequence, autonomous A → B drive	Stage 6

Ground rules

Manufacturer's libraries are not allowed

You may not use the `AlphaBot` library or the `E01`–`E12` examples supplied by Waveshare — neither through `#include` nor by copying fragments of them. The motor layer, encoder handling, odometry, the controllers and the protocol are all to be written from scratch.

There are two reasons. First, the assignment is about understanding and designing those layers, not about calling them. Second, the manufacturer's library contains bugs that matter here — speed is stored as a `char` (range −128…127), so setting a value above 127 overflows the variable, and the PID example feeds the right motor with the output of the left wheel's controller. Building on that means debugging somebody else's mistakes instead of your own controller.

Allowed: the standard Arduino libraries (`Arduino.h`, `Serial`, `avr/interrupt.h`).

Your own work

The guide describes **what** each element has to do and **how to check that it works**. It contains no ready-made implementation. The structure of the program, the choice of data types, the organisation of the control loop and the protocol format are part of the assignment.

Every stage ends with a checklist. Do not move on to the next stage until the current one is ticked off — stages 3–6 rely on numbers measured in stages 1, 2 and 4.

Libraries

The skeletons in the following stages show what you may use and which modules you have to write. They contain function signatures and contracts — **the implementation is part of the assignment**.

In an `.ino` file the `Arduino.h` header is included automatically; include the rest explicitly when you need them.

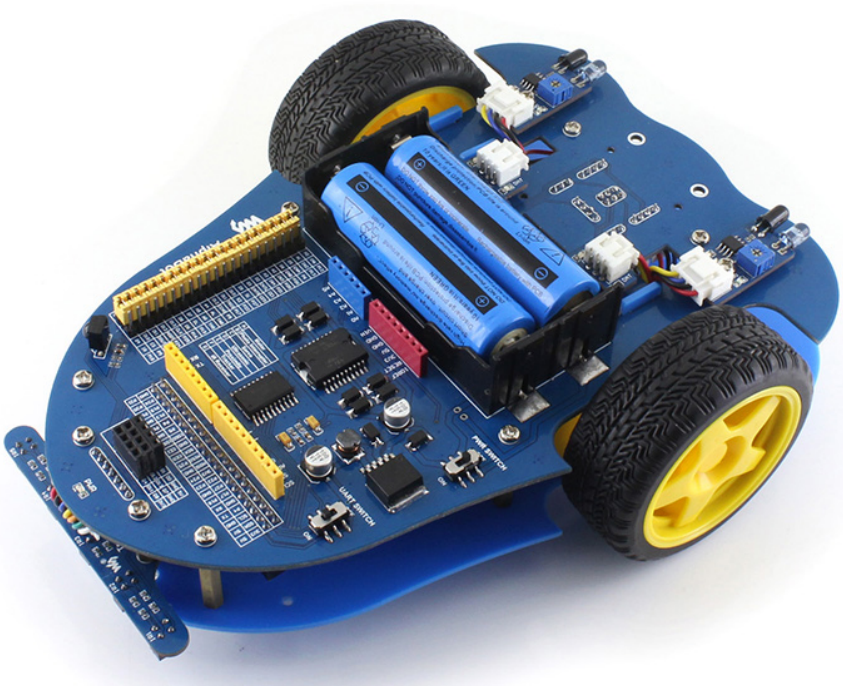
Header	What it provides	Where it helps
`Arduino.h`	`pinMode`, `digitalWrite`, `analogWrite`, `millis`, `Serial`, `attachInterrupt`	always, included automatically
`avr/interrupt.h`	interrupt macros, `cli()`, `sei()`, the `SREG` register	stage 2 — encoders
`util/atomic.h`	`ATOMIC_BLOCK` — a block executed with interrupts off and the previous state restored automatically	stage 2, as one of the ways to protect a read
`stdlib.h`	`atoi`, `atol`, `atof`, `abs`	stage 5 — converting command arguments
`string.h`	`strcmp`, `strcasecmp`, `strtok`, `strlen`	stage 5 — splitting a command line

Including `util/atomic.h` is not mandatory — it is one option among several. Choosing how to protect the counter read, and justifying that choice, is part of the assignment (stage 2).

What you must not include: `AlphaBot.h` or any file from the manufacturer's examples. The ban also covers copying fragments of their contents into your own files.

Hardware

The AlphaBot board carries a lot of peripherals. This assignment uses **three blocks**: the H-bridge that drives the motors, the wheel speed sensors and the serial interface. The remaining blocks stay unused and their jumpers have to be removed.



AlphaBot platform

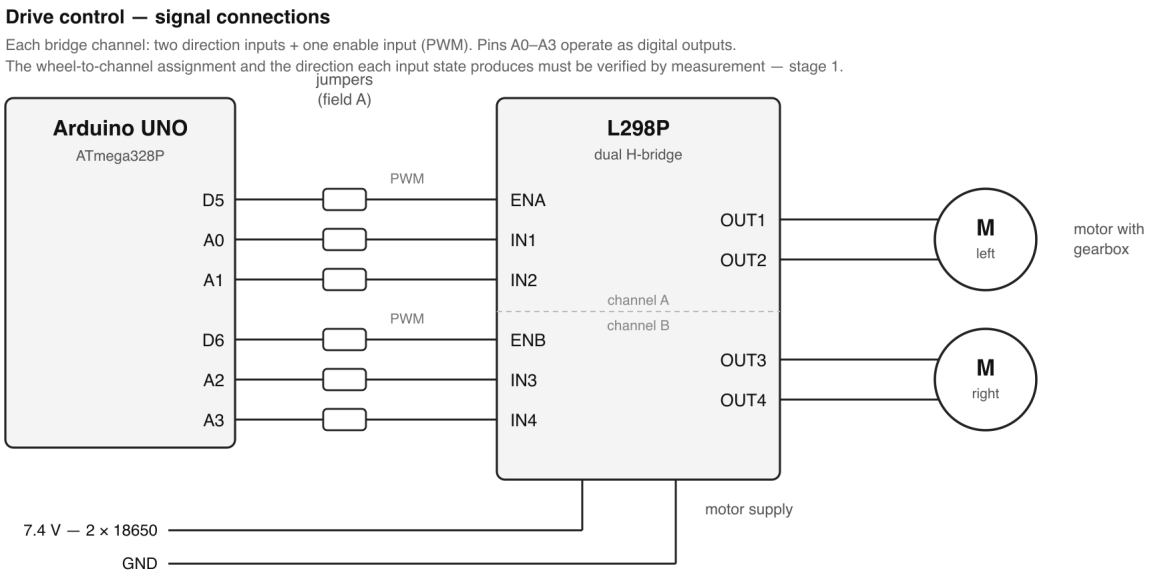
Manufacturer's catalogue photo. Visible: the main board with the holder for two 18650 cells, the driven wheels with their motors, the caster underneath, the power switch and the UART switch. The module at the front of the chassis and some of the connectors belong to peripherals this assignment does not use.

The platform is controlled **by the Arduino UNO alone**. There is no single-board computer on the robot, and every timing decision — encoder interrupts, the control loop period, serial handling — belongs to the microcontroller. That is a real design constraint: the ATmega328P runs at 16 MHz and has 2 kB of RAM, two external interrupts and one hardware UART. Every solution has to fit inside that.

The drive path

The path from the microcontroller to the wheel has four links:

1. **microcontroller outputs** — six signals, three per wheel,
2. **field A jumpers** — they select whether the signals come from the Arduino,
3. **L298P H-bridge** — it switches the supply voltage onto the windings,
4. **motor with gearbox** and the wheel with its encoder disc.



Drive signal connections

Signal	Arduino pin	Role
`ENA`	D5	channel A enable, PWM input
`IN1`	A0	channel A direction
`IN2`	A1	channel A direction
`ENB`	D6	channel B enable, PWM input
`IN3`	A2	channel B direction
`IN4`	A3	channel B direction

Notes on the table:

- pins `A0`–`A3` work as **digital outputs**, not as analogue inputs,
- `D5` and `D6` are PWM outputs — speed is set by the duty cycle on the enable input, not on the direction inputs,
- **the channel-to-wheel assignment is not assumed in this assignment.** The manufacturer's documentation allows different motor wiring and says so explicitly. Which wheel sits on which channel, and which input state corresponds to driving forwards, has to be verified by measurement in stage 1.

Pin settings for each motion

Motion	ENA	IN1	IN2	ENB	IN3	IN4
straight ahead, forwards	PWM	1	0	PWM	0	1
straight ahead, backwards	PWM	0	1	PWM	1	0
spin turn to the left	PWM	0	1	PWM	0	1
spin turn to the right	PWM	1	0	PWM	1	0
pivot about the left wheel	0	×	×	PWM	0	1
pivot about the right wheel	PWM	1	0	0	×	×
dynamic braking	?	?	?	?	?	?

coasting	?	?	?	?	?	?

Notation: **PWM** — the duty cycle that sets the speed, **1** / **0** — the logic level on a direction input, **×** — don't care (the channel is not running anyway).

The last two rows are deliberately left empty. The input states for dynamic braking and for coasting are to be derived from the L298 datasheet and confirmed by measuring the run-on distance in stage 1 — that is one of the tasks of that stage.

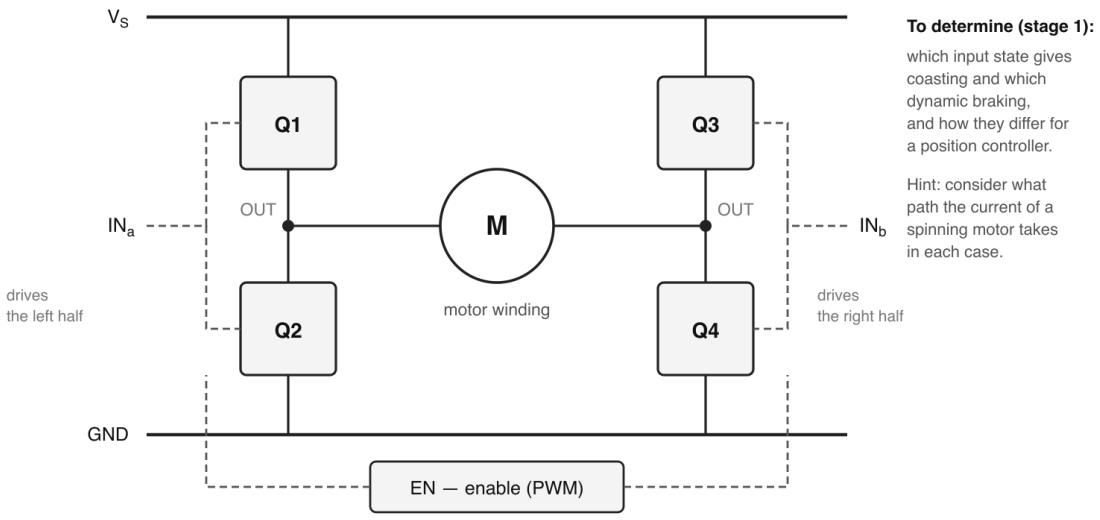
The table assumes the nominal motor wiring, with the left wheel on channel A and the right wheel on channel B. The manufacturer's documentation allows other variants, so **the table has to be confirmed by measurement before you rely on it** (direction check in stage 1). If a wheel turns the wrong way, the two values in its pair of direction inputs swap places.

The H-bridge — what you need to know about it

A single bridge channel is four switches in an H arrangement with the motor winding connected between them. Two direction inputs drive the two halves independently, and the enable input opens all the switches of the channel.

Topology of one H-bridge channel

Two halves, each driven by its own direction input. The enable input opens all switches of the channel.



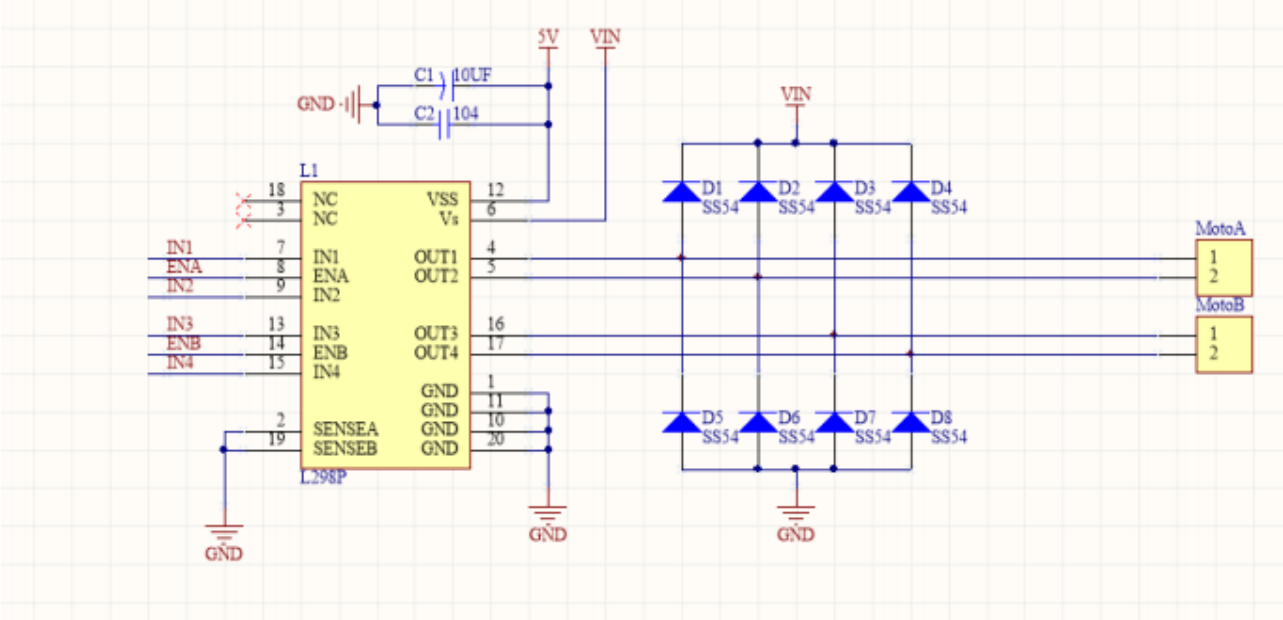
Topology of one H-bridge channel

That topology gives **four** output states, not two:

State	What happens to the winding
rotation one way	one diagonal of the bridge is closed
rotation the other way	the other diagonal is closed
coasting	outputs disconnected, winding open
dynamic braking	winding shorted through one pair of switches

The manufacturer's own state table describes only the driving directions and "stop". Telling coasting apart from braking has to be derived from the L298 datasheet and confirmed by measurement in stage 1 — by measuring the run-on distance in both cases.

Schematic of the drive block



H-bridge schematic on the AlphaBot board

The manufacturer's schematic. It shows three things the state table does not:

- **freewheeling diodes D1–D8** across the outputs. A motor winding is an inductance — when the switches change state the current cannot disappear instantly and needs a return path. Under PWM control these diodes conduct in every modulation period;
- **the `SENSEA` and `SENSEB` inputs** tied to ground. The L298 lets you insert sense resistors there and measure the motor current; this board does not use that option, so motor current is not available as a feedback signal;
- **decoupling capacitors** at the supply pins.

Note how the supplies are separated: `VIN` feeds the power stage and the outputs, while the logic of the chip runs from the 5 V rail. That is why the USB interface alone is not enough to run the drive.

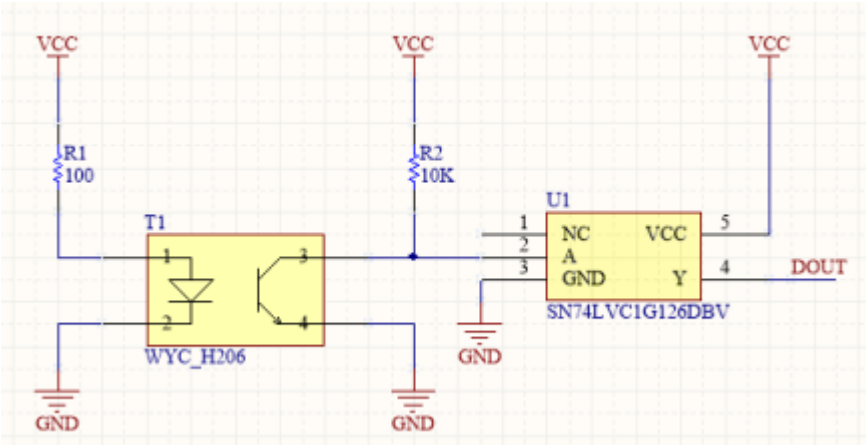
Why telling coasting and braking apart matters: a spinning motor is a generator. With the winding open its kinetic energy is dissipated only in friction; with the winding shorted it is also dissipated in the winding resistance. For the position controller of stage 3 the difference amounts to a few centimetres of travel after the stop command.

Speed sensors

An encoder disc with radial slots sits on the extension of the gearbox output shaft, on the opposite side from the wheel. A WYC-H206 slotted opto-interrupter straddles the rim of the disc: an infrared emitter on one side, a receiver on the other. The disc interrupts the beam, and the sensor output passes through a conditioning stage that produces a clean square wave.



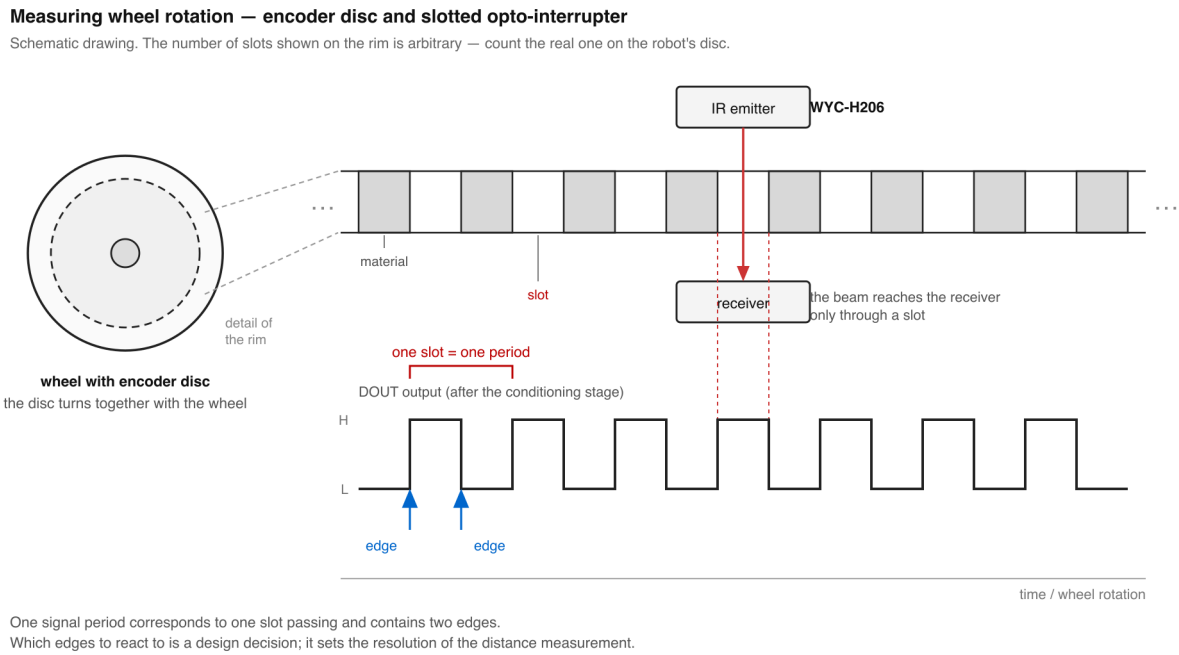
Speed sensor module



Schematic of the speed sensor module

The manufacturer's photo and schematic. The circuit is simple: the opto-interrupter, two resistors (one limiting the emitter current, one pulling up the receiver output) and an integrated circuit that conditions the DOUT output signal.

This is a good moment to compare two sources. The manufacturer's manual describes that stage as a Schmitt trigger, while the schematic shows a specific integrated circuit — read off its part number and check in the datasheet what it actually is and whether the prose description agrees. Spotting that kind of discrepancy is part of working with documentation; one sentence in the report on what came out of it is enough.



Principle of measuring wheel rotation

Signal	Arduino pin	Note
`CNTL`	D2	`INT0` — external interrupt
`CNTR`	D3	`INT1` — external interrupt

Three properties of this sensor shape the whole solution:

A single channel. The sensor gives one signal. There is no second one shifted by a quarter period, so **the direction of rotation cannot be read from the signal itself** — the counter increases the same way whether the robot drives forwards or backwards. Dealing with that is a task for stage 2.

Measurement behind the gearbox. The disc sits on the same shaft as the wheel, only on the other side of the gearbox, so it turns exactly as many times as the wheel does. The gear ratio therefore does not enter the distance calculation: one unknown fewer. The price is resolution — it is worse by exactly the gear ratio than it would be with the disc on the motor shaft ahead of the gearbox.

Check this yourself on the robot: turn the wheel through one full revolution and count the revolutions of the disc. The 1:1 assumption is the foundation of the whole odometry, so it should not remain an assumption.

Resolution is the bottleneck of this assignment. The wheel travel per count follows from the wheel circumference and the number of counts per revolution. Working it out is the first thing to do in stage 2, because it sets the limit on turn accuracy in stage 4 and a sensible stopping tolerance in stage 3.

Both external interrupts of the ATmega328P are taken. `INT0` and `INT1` serve the encoders; no other peripheral in this assignment may use external interrupts.

Encoder discs



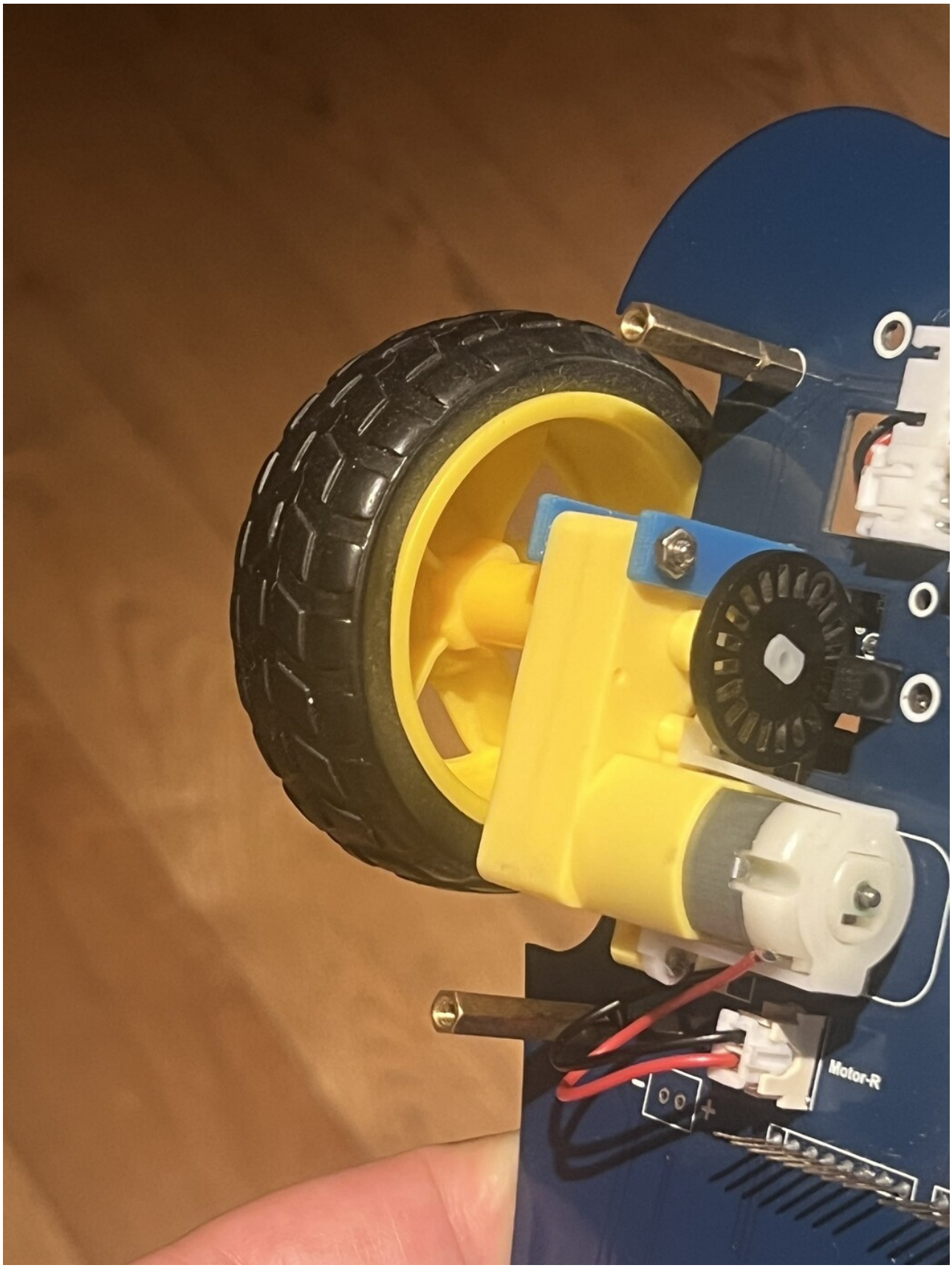
Encoder discs

Two discs in the kit, one per wheel. The central hole is shaped to match the hub, so the disc turns with the wheel without slipping. The slots are radial and evenly spaced around the circumference.

Two things to watch during assembly, because both corrupt the measurement in a way that is hard to diagnose from the program:

- the disc has to pass **inside the slot of the opto-interrupter** without rubbing against either wall. Rubbing brakes the wheel and wears the disc down over time;
- the disc has to be mounted **without play**. A loose disc vibrates when the direction changes and produces counts with no distance travelled.

The complete drive



Drive with the encoder disc

Photo of the actual setup. The whole path is visible: the wheel on the gearbox output shaft, the yellow gearbox housing, the DC motor behind it, and on the far side of the gearbox — **the encoder disc on the extension of that same shaft**, inside the slot of the opto-interrupter screwed to the gearbox housing. The motor leads go to the board connector labelled with the channel name.

What to look at when checking the assembly:

- the disc passes through the opto-interrupter slot **without touching** either wall,
- the opto-interrupter is screwed down firmly — a loose one shifts relative to the disc and starts losing pulses when the robot is jolted,
- the motor leads go to the connector matching the correct side of the robot. Swapping the left and right motor leads produces a symptom that is easy to mistake for a software bug: the robot turns the wrong way even though driving straight looks correct.

To be counted before starting stage 2

Count the slots on the encoder disc. That number is deliberately not given in this guide — it enters the formula for the travel per count, so an error in it propagates into the whole odometry.

How: remove the disc, or mark a reference point on the rim and count the slots while going around the disc. **Verify the result electrically**: turn the wheel by exactly one revolution and compare the number of counts with your prediction. Agreement between the two methods confirms that both the disc and the interrupt configuration are what you assume them to be.

Do the count for **both** wheels — do not assume the discs are identical.

Serial interface

The Arduino UNO has one hardware UART, brought out through a USB converter to the connector. The same link carries both program upload and the protocol from stage 5, so the port can be held by only one tool at a time.

The UART receive buffer in the Arduino library has a limited capacity. Look its value up in the documentation — it is needed in stage 5 to decide the flow control rule.

Power

Two 18650 cells in series give 7.4 V. The voltage goes through the on-board converter to the logic and directly to the bridge as the motor supply.

The drive is not powered from USB. With the power switch off or the cells flat, the Arduino runs, answers over serial and looks healthy while the robot stands still. That is the first hypothesis for the symptom "it does not move".

Take all calibration measurements at a similar state of charge and note the voltage. A drop from 8.4 V to 7.0 V changes the speed at the same PWM duty by more than ten per cent, which invalidates the ramps and dead-band thresholds you have chosen.

Check before first power-up

- ☐ encoder discs fitted, inside the opto-interrupter slots, not rubbing
- ☐ speed sensors plugged in, each to its own wheel
- ☐ cells fitted according to the polarity marking, voltage noted
- ☐ slots counted on both discs

Stage 1 — Bringing up the drive, straight line and turning

Goal

A working open-loop motor layer, and a feel for how the robot behaves without control.

What to implement

Motor layer. A function that sets the speed of one wheel, taking a signed value covering the full PWM range in both directions. The function sets the direction inputs according to the sign and puts the magnitude on the enable input.

Mind the data type: the range has to cover full PWM in both directions. This is exactly where the manufacturer's library has the bug described at the start of the guide.

Stopping functions. Coasting and dynamic braking separately — for the comparison in the measurement below.

Three motion modes, as combinations of the two wheel speeds:

Mode	Description
straight	both wheels the same way
spin turn (*spin*)	wheels counter-rotating, turn about the centre of the track
pivot turn (*pivot*)	one wheel stationary, turn about it

The *pivot* mode will be useful in stage 4 — worth having from the start.

Direction check

Put the robot on a stand (wheels in the air) and check each mode separately, in both directions. Confirm which way each wheel turns.

If a wheel turns the wrong way, apply the correction **in the code** — a constant that inverts the sign for that wheel — and write it down. A correction in the code is explicit and gets documented; re-plugging the motor connectors does not.

Measurements

PWM dead band. For each wheel separately, find the smallest PWM duty at which the wheel turns continuously (without stuttering). Take the measurement with the wheels **on the floor** — under load the threshold is higher than on a stand.

For the later stages adopt a single value: the larger of the two measured, increased by a margin of about 20 %. The reason will become clear in stage 3 — the controller subtracts a correction from one wheel's PWM, so it needs headroom above the threshold.

If the two thresholds differ by more than about 25 %, check the mechanics (a rubbing encoder disc, play in the gearbox) before going further.

Run-on distance. For two PWM values — medium and high — measure how far the robot travels after the drive is removed. Separately for coasting and for dynamic braking.

The result of this measurement answers the question of why, in stage 3, the condition "target reached → stop" is not enough on its own.

Checkpoint 1

- ☐ The robot drives straight forwards and backwards
- ☐ It spins in place in both directions
- ☐ It performs a *pivot* turn in both directions
- ☐ Both wheel directions confirmed by measurement, not assumed; corrections noted
- ☐ Dead band measured for each wheel, working value adopted with a margin
- ☐ Run-on distance measured for coasting and braking, at two PWM values

Skeleton — motor layer

```
#include <avr/interrupt.h>

/* FORBIDDEN: #include "AlphaBot.h" and any file from the manufacturer's E01-E12 examples */

/* ===== interface speeds =====
   All link speeds as named constants, in one place at the top of the file.
   An inconsistency in this single number between the two ends of the link
   produces the symptom "nothing works" with completely correct code.          */
static const unsigned long SERIAL_BAUD = /* TODO */;

/* ===== pin map ===== */
static const uint8_t PIN_ENA  = 5,  PIN_IN1 = A0, PIN_IN2 = A1;  /* channel A    */
static const uint8_t PIN_ENB  = 6,  PIN_IN3 = A2, PIN_IN4 = A3;  /* channel B    */
static const uint8_t PIN_CNTL = 2,  PIN_CNTR = 3;                /* encoders     */

/* ===== motor layer =====
   The only place in the program that touches the bridge pins. The rest of the
   program sets a signed speed and knows nothing about direction input states.  */

void motorL(int pwm);   /* sign = direction, magnitude = PWM duty               */
void motorR(int pwm);   /* range wide enough for full PWM both ways             */
void brake();           /* dynamic braking                                      */
void coast();           /* coasting                                             */

/* TODO: in motorL/motorR derive the direction input states from the sign of the
         argument and put the magnitude on the enable input. Provide for the motor
         orientation correction found by measurement (direction check below).
   TODO: in brake/coast establish the input states for both ways of stopping
         - from the L298 datasheet and the topology drawing in the hardware
         chapter.                                                             */

void setup() {
  /* --- 1. pin directions ---
     After a reset every pin is an input. Until they are switched over, the
     bridge inputs see an undefined state.                                    */

  pinMode(PIN_ENA, OUTPUT);     /* a bridge input = an OUTPUT for the MCU      */
  /* TODO: the remaining five bridge pins (IN1, IN2, ENB, IN3, IN4) likewise   */

  pinMode(PIN_CNTL, INPUT);     /* a sensor output = an INPUT for the MCU      */
  /* TODO: the second speed sensor channel                                    */

  /* Note: A0-A3 set as OUTPUT work as ordinary digital outputs, even though
     their default role is an analogue input.                                 */

  /* --- 2. safe initial state ---
     TODO: stop the drive before the program does anything else. A robot standing
           on the floor must not twitch at start-up or on reset.              */

  /* --- 3. serial link --- */
  Serial.begin(SERIAL_BAUD);
  /* TODO: one greeting line with the name of the loaded program - with several
           test sketches around it saves confusion about which one is in memory. */
}

void loop() {
```

```
  /* TODO: read a single character from the terminal, select the motion mode,
           confirm with one line. No printing inside the loop - the result has to
           be copyable from the terminal.                                        */
}
```

The order inside `setup()` is not arbitrary: pin directions first, then the safe state of the drive, and only then the link. The reverse order means the bridge inputs stay undefined for the whole time the link is being initialised.

Stage 2 — Wheel distance and speed from the encoders

Goal

A signed distance counter for each wheel, and a calibrated coefficient converting counts to millimetres.

Background

Counts per revolution

Let S be the number of slots in the disc — **counted on the robot**, as instructed in the hardware chapter. The opto-interrupter is blocked and unblocked in turn, so one wheel revolution is S signal periods, and each period contains two edges.

The number of counts per wheel revolution depends on the interrupt configuration:

$$N_rev = S \text{ (reacting to one edge)}, \quad N_rev = 2S \text{ (reacting to both edges)}$$

The choice is a design decision. Resolution is the bottleneck of this assignment — it is worth thinking through what happens to the accuracy of a small turn (stage 4) under each variant before making the decision.

Converting to distance

$$k = (\pi D)/N_rev$$

where D is the wheel diameter and k the travel of the wheel circumference per count.

The manufacturer **does not give** the wheel diameter in the documentation. Measure it with callipers, but derive the working value by calibration over a distance — and use that one.

The resolution of the distance measurement, equal to k, comes out at a few millimetres. That means the distance measurement is quantised, and no position controller can have a stopping tolerance smaller than one or two counts. The consequences for turning are in stage 4.

The direction problem

The opto-interrupter has **one channel**. There is no second, phase-shifted signal, so **the direction of rotation cannot be read from the sensor output** — the counter increases equally when driving forwards and backwards.

The solution has to be designed by you. The questions the design must answer, and which are worth discussing in the report:

- where to get the sign of the distance from, if the sensor does not supply it,
- what should happen to the counter when the robot is standing still and the disc vibrates,
- what should happen to the counter **while coasting**, when the drive is already off but the robot is still moving (the run-on measurement from stage 1 shows how much that is).

The last two points are in conflict: a solution that cuts the counts off as soon as the drive stops loses the coasting distance; one that does not cut them off collects counts from vibration. The compromise is part of the assignment.

Interrupt service routine

The interrupt service routine should be as short as possible — no arithmetic, no printing to serial. All conversions belong in the main loop.

Variables shared between the interrupt and the main loop need two things: the right type qualifier and an **atomic read** in the main loop. The ATmega328P is 8-bit, so reading a variable wider than a byte can be interrupted halfway and return a value assembled from two different states. How to protect the read is for you to decide; note that the mechanism must not enable interrupts in a situation where they were disabled before the call.

Measuring speed

Two approaches, to choose between and justify:

Method	Principle	Behaviour
counts in a time window	counts divided by the window length	noise grows at low speed, because the window holds few counts
period between edges	the reciprocal of the time between counts	accurate at low speed, sensitive to jitter, needs handling for "no counts"

Work out the count rate at a typical driving speed, substituting your own *N_rev* and wheel circumference. Then work out how many counts fall inside a window as long as the control loop period, and draw a conclusion about the window length and about which of the two methods suits what.

Position control in stages 3 and 4 does not need speed — the controller works on position error and the difference of wheel distances. Speed is needed for the characteristic below.

Checks before calibration

Three tests, each catching a different typical fault:

Test	How	Correct result
both channels count	turn one wheel by hand through several revolutions	that wheel's counter rises, the other stays put
counts per revolution	turn a wheel by hand through exactly 10 revolutions	10 revolutions according to the conversion
sign of the distance	drive forwards, then backwards	positive and negative values respectively

A second-test result equal to half the expected value means the interrupt fires on one edge; twice the expected value means double counts from vibration or a dirty disc.

Measurements

Calibration of k. Mark out a section on the floor (at least 1 m is recommended), place the robot with its **wheel axis** over the start line, drive and stop exactly on the finish line, read off the counts. Three repetitions plus one in reverse.

Derive *k* from the measurement, compare it with the value computed from the measured wheel diameter, and explain the difference. A spread between runs above about 2 % points to a flaw in the method, not in the hardware.

Static PWM → speed characteristic, for each wheel separately, for at least five PWM values across the useful range. Two things needed later follow from it:

- the operating point for cruising — inside the linear part of the characteristic,

- the **wheel asymmetry**, that is the speed difference at the same PWM. This is the quantity the controller of stage 3 has to compensate; note it down, because the larger it is, the more clearly the controller's effect shows.

If the asymmetry exceeds about 15 % or changes sign across the PWM range, check the mechanics — the controller will not fix that.

Code to write

Process the measurements: derive *k* with its mean and spread, compare with the wheel geometry, plot both characteristics and the asymmetry. The cells below are empty — the data-processing code is part of the assignment.

Skeleton — odometry module

Sensor pulses can be counted in two ways. Both are acceptable; **the choice and its justification are part of the assignment**.

	variant A: interrupt	variant B: polling in the loop
completeness of counts	loses no pulses, however long one pass of the main loop takes	loses pulses if one pass takes longer than half the shortest signal period
shared variables	the counter is modified outside the main loop — needs the right type qualifier and a protected read	ordinary variables, the problem does not arise
resources	takes `INT0` and `INT1`, both external interrupts	takes no interrupts
sensitivity to blocking	immune to blocking sections of the program	any operation that stalls the loop (e.g. printing to the link) risks losing a pulse
complexity	higher	lower

To work out before choosing: the maximum pulse rate at the highest speed from the characteristic measured in this stage, and the longest permissible loop pass that follows from it for variant B. Compare the result with what the main loop will be doing in stage 6, once command reading and the controller are added to it.

```
/* ===== counter state =====
   TODO: declare the counters for both wheels and whatever the module needs to know
         about the direction of travel - the sensor does not supply that.
         For variant A pay additional attention to:
         - the type qualifier for variables modified outside the main loop,
         - the type width given an 8-bit core and the range of distances here.   */

/* ===== variant A: counting in an interrupt ===== */

void isrL();    /* interrupt service routine: as short as possible, no floating  */
void isrR();    /* point arithmetic and no printing to the link                  */

/* in setup():
   TODO: attachInterrupt(digitalPinToInterrupt(PIN_CNTL), isrL, <edge mode>);
         The edge mode is a design decision - it changes the counts per revolution
         and through that the distance resolution and the turn accuracy in stage 4. */

/* ===== variant B: counting by polling ===== */

void encoderPoll();   /* called on EVERY pass of loop(), never less often         */

/* TODO in encoderPoll():
     - read the state of both inputs,
     - detect a change against the state from the previous call (the previous state
       has to be remembered somewhere),
```

```
     - react to the same edges you would have chosen in variant A.
   Note: the function must not wait for anything - its execution time is part of
         the loop period that it itself constrains.                          */

/* ===== access from the main loop (common to both variants) ===== */

long  readTicksL();            /* in variant A the read must survive an interrupt
                                  in the middle; in variant B it need not       */
long  readTicksR();
void  odoReset();              /* zero the counters                             */
float ticksToMm(long ticks);   /* needs the constant found by calibration        */
```

When writing `readTicksL`/`readTicksR` in variant A, decide what should happen if the function is called from a context where interrupts are already disabled. A solution that unconditionally enables them at the end behaves differently there from what the caller expects.

Separating `readTicks*` from the rest of the program has one more advantage: the choice of variant stays inside the odometry module and does not leak into the controllers of stages 3 and 4.

```
# Calibration of k [mm/count] from the drive measurements.
#
# Input  : measured distance, measured wheel diameter, counts from each run
# Output : mean k and spread, the value from wheel geometry, a comment on the difference,
#          distance measurement resolution, wheel asymmetry
```

```
# Static PWM -> speed characteristic for both wheels.
#
# Input  : table of (PWM, left wheel speed, right wheel speed)
# Output : plot of both characteristics, linear range, chosen operating point,
#          plot of asymmetry against PWM
```

Checkpoint 2

- ☐ Both channels count, hand-turn test performed
- ☐ Counts per wheel revolution confirmed
- ☐ The counter changes sign when driving backwards
- ☐ Documented decision on how the sign of the distance is derived and how coasting is treated
- ☐ Counter read in the main loop protected; justification in the report
- ☐ *k* derived from at least 3 runs, spread below 2 %
- ☐ Difference between calibration and wheel geometry explained
- ☐ PWM → speed characteristic for both wheels, at least 5 points
- ☐ Operating point chosen, wheel asymmetry noted

Stage 3 — Straight-line controller for a given distance

Goal

The command "drive the given number of millimetres forwards or backwards", executed in a feedback loop closed by the encoders.

Background

Two controllers, two different jobs

The straight-line controller consists of two independent parts acting on the same pair of motors:

The distance controller answers the question *how fast to drive* given how far is left to the target, and when to stop. It is not a proportional controller — it is a speed profile generator with a stopping condition.

The symmetry controller answers the question *how much to differentiate the wheels* in order to drive straight. This is the one required by the assignment: a proportional controller based on the difference of the left and right wheel odometry.

Quantities

Let N_L, N_R be the counts since the start of the motion and k the coefficient from stage 2.

Progress along the path:

$$s = (N_L + N_R)/2\ k$$

Heading error expressed as the difference of the wheel distances:

$$e = (N_L - N_R)\ k$$

For a differential drive with track b that difference is tied to the change in orientation:

$$\Delta\vartheta = (s_R - s_L)/b$$

So by driving e to zero you hold the orientation from the moment of the start — the robot drives straight.

A note on interpretation: the controller zeroes the *orientation*, not the *lateral deviation*. A robot that has drifted off and straightened up will carry on straight, but parallel to the intended direction, with the offset it has accumulated. That is why it matters that the error should not build up, rather than that it should be corrected after the fact.

Control law

A correction proportional to the difference of counts, added to one wheel and subtracted from the other. Unit of the gain: PWM units per count of difference.

To be designed:

- how to get the sign of the correction right when **driving backwards**, where both counters are negative, without branching the code into two cases,
- what to do when the correction is so large that it would push one wheel's PWM below the dead band from stage 1. Consider what physically happens to that wheel then, and whether the controller is still improving what it was meant to improve.

Speed profile

A ramp for acceleration and deceleration is needed, driven by position rather than time.

Why it is necessary:

- **at the start** — applying PWM as a step makes the wheels slip, and slip means counts with no distance travelled, that is an odometry error no controller can detect;
- **at the end** — the run-on distance measured in stage 1 shows how far the robot travels after the drive is removed. Slowing down before the target reduces that distance.

Choose the ramp length from the run-on measurement.

Note the side effect: during the ramp the PWM is close to the dead band, so the headroom for the symmetry correction is small or zero. Lengthening the ramp improves the smoothness of the start and worsens the heading correction there. That is a genuine trade-off — worth measuring and describing.

Choosing the gain

There is no formula. The starting reasoning: decide at what error (in counts) the correction should reach some fraction of the cruising PWM, and compute the gain from that. When choosing both numbers, take into account the encoder resolution from stage 2 and the headroom above the dead band.

Then tune on the robot. It is worth providing for changing the gain **by a command from the terminal**, without recompiling — tuning by uploading the program every time is incomparably slower.

Observations that will come up during tuning, and what each one means:

Symptom	What it points to
the robot drives straight but one wheel stutters	the correction pushes the wheel below the dead band
visible weaving	the gain is too high for the encoder resolution
lateral deviation stops improving as the gain rises	saturation — a mechanical problem, not a control one
the robot always drifts to the same side regardless of gain	a difference in wheel diameters (a scale error), not drive asymmetry

Do not add an integral term until the proportional controller is tuned. For a constant one-sided error it is better to correct the wheel scale (the last row of the table). An integral term added without anti-windup means that after stopping the robot "remembers" the accumulated error and starts the next motion with a turn.

Measurements

Document the tuning: at least four gain values, with the lateral deviation measured for each. Zero gain as the reference — it shows how much the controller has to make up for.

Series of runs over a fixed distance, in both directions. For each run note three numbers: the commanded distance, the distance according to odometry and the actual distance measured with a tape, plus the lateral deviation from the straight line.

Those three numbers give **two different errors**, which are often confused:

Error	Definition	What it tells you
controller error	odometry − commanded	whether the robot reached its own target
scale error	actual − odometry	the quality of the k calibration from stage 2

The first improves with the ramp and the stopping tolerance, the second **only** with a correction to *k*. Separating them is the point of this measurement — the robot can reach its odometric target perfectly and still miss the physical one.

It is also worth recording the symmetry error against time for two gain values: the chosen one and one that is too high. That needs a telemetry stream from the robot — provide for switching it on and off by a command, and remember that telemetry inside the control loop competes for time with the control itself.

Code to write

Skeleton — straight-line controller

```
/* ===== parameters from the calibration sheet =====
   Values found by measurement, not copied from the documentation.              */
static float mmPerTick = /* TODO: stage 2 */;
static int   pwmCruise = /* TODO: stage 2 */;
static int   pwmMin    = /* TODO: stage 1 */;
static float kpSync    = /* TODO: stage 3 */;

/* ===== straight-line controller =====
   Split into three functions, because the motion takes seconds and must not block
   the main loop.                                                               */

void driveStart(long mm);   /* record the initial state and the target; mm signed  */
void driveStep();           /* ONE control step, called at a fixed period          */
bool driveDone();           /* has the motion finished                             */

/* TODO in driveStep():
     - progress derived from both counters,
     - stopping condition with a tolerance matched to the encoder resolution,
     - speed profile as a function of position, not time,
     - symmetry correction proportional to the difference of the wheel odometry,
     - limit on the correction so that neither wheel falls below the dead band,
     - correct sign of the correction when driving backwards too, without branching. */

/* ===== fixed-period loop ===== */
static const unsigned long CTRL_DT_MS = /* TODO */;
```

Two triggering variants, differing by one line:

```
static unsigned long tCtrl = 0;

void loop() {
  unsigned long now = millis();
  if (now - tCtrl >= CTRL_DT_MS) {
    /* variant 1:  tCtrl = now;        */
    /* variant 2:  tCtrl += CTRL_DT_MS; */
    /* TODO: pick one and justify it. They differ in whether a delay in a single
             pass is made up for. For the controller that means a fixed or a varying
             sampling period - and the effective gain depends on that period.     */
    driveStep();
  }
  /* TODO: the rest of the loop - nothing here may stall execution               */
}
```

The form `now - tCtrl >= CTRL_DT_MS` on unsigned variables also works correctly at the moment the `millis()` counter overflows. The form `now >= tCtrl + CTRL_DT_MS` does not have that property.

```
# Processing a series of runs.
#
# Input  : for each run (commanded, odometry, actual, lateral deviation)
# Output : controller error and scale error separated, with mean and deviation;
#          corrected value of k; conclusion on which error dominates
```

Checkpoint 3

- ☐ Driving a given distance works in both directions, for several distances
- ☐ The ramp works — the robot neither jerks at the start nor overshoots the target
- ☐ The correction-versus-dead-band limit solved; the decision justified
- ☐ Tuning documented: at least 4 gain values with the symptoms described
- ☐ Lateral deviation measured for zero gain and for the chosen gain
- ☐ Symmetry error recorded against time for two gain values
- ☐ Controller error and scale error separated; k corrected

Stage 4 — Turn controller for a given angle

Goal

The command "turn by a given angle to the left or to the right", with the accuracy required by the assignment (at least 45°), and measured accuracy for 15, 30, 45 and 90°.

Background

Kinematic model

For a differential drive with track b:

$$\Delta\vartheta = (s_R - s_L)/b$$

From this formula, derive the distance a wheel has to travel to produce a given turn — separately for the spin turn and for the pivot turn. The two results differ by a factor of 2.

Put the derivation in the report; the conclusion about resolution below follows from it.

Sign convention

Adopt and **record in the protocol description** (stage 5) an unambiguous convention: which direction of rotation corresponds to a positive angle. Mixing conventions is the most common source of errors when assembling the command sequence in stage 6.

The wheel track is a calibration parameter

The track measured with a tape is not the same quantity as the one in the formula. The reasons:

- **tyre width** — the contact point is not a point; during a turn the tyre twists and the effective contact point shifts,
- **lateral slip** — in a spin turn the tyres have to slide,
- **the caster** — its friction when the direction changes introduces a moment that moves the centre of rotation.

That is why the track has to be **calibrated from an angle measurement**, not measured.

Angular resolution — the key limitation of this assignment

One encoder count corresponds to a distance k along the wheel circumference. From the kinematic model, derive **the smallest change in orientation the controller is able to notice** — separately for the spin turn and for the pivot turn. Work those values out for your own robot, substituting k from stage 2 and the track from the calibration below.

Then work out how many counts per wheel a 15° turn amounts to, and compare that with the stopping tolerance.

The conclusion from this calculation matters for the whole assignment: the 45° required by the assignment is achievable without difficulty, while 15° lies close to the resolution limit of the sensor. Ways to improve it, to consider and justify your choice:

Direction	What it involves
choice of turn mode	the two modes from stage 1 have different angular resolution; which one and by how much follows from the derivation
lower speed for small angles	less run-on, a more accurate stop; the risk of sticking in the dead band

closing the loop through the operator	the robot reports the **achieved** angle and the operator adds a correcting command; how far that converges and why is for you to establish

The last point requires the response to a turn command to carry the angle actually achieved according to odometry — the result line from stage 5 has to contain that value.

What to implement

A turn controller analogous to the straight-line one: a speed profile as a function of progress, a stopping condition, braking, and a **symmetry controller** — here equalising the magnitudes of the two wheel distances rather than their difference.

Note that in the pivot mode the definition of progress differs from the spin mode, and the symmetry controller does not apply there.

The response at the end of a turn has to carry the angle achieved according to odometry.

Measurements

Track calibration. Command a turn through a multiple of a full revolution — five is recommended — and measure the final deviation from a reference mark with a protractor. The track correction follows from the ratio of the actual angle to the commanded one.

Why a multiple rather than a single revolution: the protractor reading error is then spread over five times the angle. Repeat the measurement in both directions; a difference in the sign of the correction points to asymmetric friction of the caster.

Turn accuracy. For 15, 30, 45 and 90°, in both directions, five repetitions each. Note the commanded angle, the angle achieved according to odometry and the angle measured with a protractor.

Repeat for the other turn mode — at least for the small angles. The comparison of the two modes is the strongest argument in the report, because it shows whether the limitation is one of resolution or one of control.

Interpreting the results:

Observation	Cause
the mean error grows in proportion to the angle	the track is under-calibrated
the mean error is constant, independent of the angle	run-on — lengthen the ramp or lower the turn speed
the spread does not shrink under any tuning	the resolution limit of the sensor

Code to write

Skeleton — turn controller

```
static float trackMm = /* TODO: stage 4, from calibration - not from a tape measure */;

void  turnStart(float deg, bool pivot);  /* angle sign per the adopted convention   */
void  turnStep();
bool  turnDone();
float turnAchievedDeg();                 /* angle achieved per odometry             */

/* TODO in turnStart(): the wheel distance corresponding to the commanded angle. The
        formula differs between the two turn modes and follows from the kinematic
        model - derive it, do not guess.
   TODO in turnStep(): the definition of progress depends on the mode; the symmetry
```

```
        controller applies to only one of them - decide which and why.            */
```

`turnAchievedDeg()` is not decoration: in stage 4 it lets you establish whether the limitation is the sensor resolution, and in stage 6 it lets the operator pick a correction in phase 1. The result line printed at the end of a turn has to contain that value.

```
# Wheel track calibration from a multi-revolution measurement.
#
# Input  : number of full revolutions, final deviations for both directions
# Output : calibrated track, comparison with the tape measurement, left/right asymmetry
```

```
# Angular resolution of your own robot.
#
# Input  : k from stage 2, track from the calibration
# Output : smallest detectable change of orientation for both turn modes,
#          counts per wheel for 15/30/45/90 degree turns,
#          relative angle uncertainty for each of them
```

```
# Turn accuracy - processing the measurement series.
#
# Input  : for each angle and direction a list of measured angles (5 repetitions)
# Output : mean error and spread for each angle, comparison of the two turn modes,
#          conclusion: is the limitation one of resolution or of control
```

Checkpoint 4

- ☐ Derivation of the wheel distance for a given angle, for both turn modes
- ☐ Angle sign convention adopted and recorded
- ☐ Track calibrated by the multi-revolution procedure, in both directions
- ☐ Difference against the tape measurement explained
- ☐ Angular resolution computed for both modes, for your own robot
- ☐ Table of 15/30/45/90° × left/right × 5 repetitions
- ☐ The same for the other turn mode, at least for the small angles
- ☐ The 45° accuracy required by the assignment achieved
- ☐ Discussed why 15° has a larger relative error and how to work around it
- ☐ The response to a turn command carries the achieved angle

Stage 5 — Communication protocol over the serial interface

Goal

The robot is to execute a route given as a single string sent over the serial interface.

Command format

The operator sends a sequence of commands separated by spaces. Each command is a **command letter** followed by a **number**:

```
G1.5 L45 R-45 G2
```

That means: drive 1.5 m straight, turn 45° left, turn 45° right, drive 2 m straight.

Letter	Meaning	Argument
`G`	drive straight	distance in metres, signed (negative = backwards)
`L`	turn left	angle in degrees
`R`	turn right	angle in degrees

Characters other than a command letter are **ignored**. Spaces, line endings and anything the robot does not recognise simply trigger no action. That way there is no need to check whether the operator separated the commands with a space, a comma, or sent them over several lines.

To be decided: in the example `R-45` means a 45° turn to the right, so the direction is carried by the command letter, not by the sign of the number. You have to decide and record what the program does with a negative number after `L` or `R` — the simplest solution is to take the absolute value, so that the direction follows from the command letter alone. Describe the rule you adopt in the report.

Reading

Two functions of the `Serial` library are enough:

Function	Returns
`Serial.read()`	one character from the input buffer
`Serial.parseFloat()`	a number read from the following characters of the buffer

The scheme: take a character; if it is a command letter, read the number that follows and execute the command; otherwise skip the character and carry on reading.

Units and conventions

Decide and record:

- the distance unit in the `G` command — in the example it is metres, while the odometry works in millimetres, so a conversion has to happen somewhere;
- what a negative value means for `G` and for `L` / `R`;
- the permissible ranges and the behaviour when they are exceeded.

Confirming execution

After each executed command the robot prints **one line** with the result: what was commanded and what was achieved according to odometry. For a turn that line has to contain the angle actually achieved — that is the value computed in stage 4 and needed to assess accuracy.

The result lines are the only thing the robot prints. Print nothing inside the control loop: the result of a drive has to be copyable from the terminal.

Checkpoint 5

- ☐ The robot executes a command sequence sent as a single string
- ☐ Unrecognised characters are skipped without side effects
- ☐ The rule for negative values and the units decided and recorded
- ☐ Behaviour defined for an out-of-range argument
- ☐ One result line after each command; for a turn it contains the achieved angle
- ☐ The whole protocol can be exercised from the serial monitor, with no program on the computer

Skeleton — reading commands

```
void handleCommands();   /* called on every pass of loop(), non-blocking */

/* Outline:

   while (Serial.available() > 0) {
     char ch = Serial.read();

     switch (ch) {
       case 'G':
         // TODO: read the number with Serial.parseFloat() and drive straight
         break;

       case 'L':
         // TODO: read the number and turn left
         break;

       case 'R':
         // TODO: read the number and turn right
         break;

       default:
         break;          // unrecognised character - skip it, do nothing
     }
   }

   TODO to decide:
     - converting the command unit into the odometry unit,
     - treatment of a negative value after L and R,
     - behaviour for an out-of-range argument,
     - printing the result line once the command has finished.                */
```

Why `switch` rather than a chain of conditionals:

- the decision is in one place and all the command letters are visible at once; adding another command is one more case, not another branch tacked on at the end,
- the `default` branch is **explicit handling of unrecognised characters**, which is exactly what the protocol requires — with a chain of conditionals, skipping a character is merely a side effect of no condition having matched,
- lower and upper case can be handled by one case: `case 'G': case 'g':` — whether the protocol should distinguish them is for you to decide.

Mind the `break` at the end of each case. Omitting it falls through to the next case, which with this protocol means executing a different motion from the one intended — and the compiler does not warn about it by default.

`Serial.parseFloat()` waits for characters until it has assembled a number or its own timeout expires. In the stage 5 test sketch that is acceptable. In stage 6, where a motion takes seconds, you have to decide what happens to commands that arrived in the meantime — and whether the whole string should be read before the drive starts or during it.

Stage 6 — Autonomous A → B drive

Goal

The acceptance scenario from the assignment. Nothing new to write beyond merging stages 1–5 into one working program.

Procedure

Phase 1 — establishing the route. The robot stands at point A, on the start orientation mark. The operator types **single commands** into the serial monitor — `G0.8`, `L90`, `G1.2` and so on — watches the robot and picks the next one, until it comes to rest at point B. Every command used is written down.

Phase 2 — autonomous drive. The robot goes back to point A, onto the same mark. The operator sends **one string** made up of the recorded commands, for example:

```
G0.8 L90 G1.2 R45 G0.6
```

The robot executes the whole route with no further intervention.

Phase 3 — measurement. The deviation of the wheel axis from point B and the heading deviation. Repeat phases 2 and 3 three times, without repeating phase 1, and compute the spread.

How to record the route in phase 1

The command sequence has to be the one that will actually be sent in phase 2 — not a log of everything the operator typed along the way. Two things to decide:

- **corrections.** If after `G1.0` it turned out that another `G0.3` was needed, the sequence should contain `G1.3`, not both commands. Two consecutive turns the same way add up as well;
- **reversals.** If the operator overshot and backed up, the difference goes into the sequence. Not every such correction can be merged — a turn performed between two straight sections changes the geometry and cannot be folded in.

Establishing the route this way is slower than driving with a joystick, but it has an advantage: the command sequence emerges directly in the form in which it will be sent, with no intermediate conversion step.

What can go differently in phase 2 than in phase 1

This is the heart of the stage and the material for the conclusions. The same commands, a different result — why:

Cause	Why it works that way
the string does not fit in the input buffer	some commands are lost; see the calculation below
commands executed back to back with no pause	in phase 1 there was a pause between commands, in phase 2 there is none — a different initial state for each motion
the cells losing voltage	phase 2 happens later than phase 1
a different starting position	the orientation mark at point A has to be reproduced exactly
accumulation of heading error	every turn contributes an error of the order of the quantum from stage 4, and that is amplified by the length of the remaining route

To work out: the length of the string for your own route in characters, compared with the capacity of the UART input buffer in the Arduino library. The result decides whether the whole string can be sent at once or has to go in batches — and which of the command execution variants in this stage is available to you.

Architecture

The program is an assembly of what was built earlier:

Block	From	When it runs
odometry	stage 2	in an interrupt or by polling
drive and turn controllers	stages 3 and 4	at a fixed period
command reading	stage 5	on every pass of the main loop

Decide which block is to be serviced first and why. Whatever you choose, **a function that stalls execution must not appear in the control loop**: a "drive for a given time" construction built on such a function makes any reaction during the motion impossible. The manufacturer's examples are built exactly that way; that is one more reason for the ban at the start of the guide.

Preparing the test area

5. Mark points A and B on the floor — at least 2 m apart, with at least one change of direction between them.
6. Mark the **start orientation** of the robot at point A. Without it the drive is not repeatable, and phase 2 has no chance of ending where phase 1 did.
7. Charged cells, voltage noted. The parameters from the calibration sheet entered.

Analysing the result

Separate the two components of the arrival error:

Component	Symptom	What to fix
systematic	the mean shifted in one direction, repeatable	calibration: `mmPerTick` for an error along the route, the wheel track for an error across it
random	a large spread with a small shift of the mean	the limit of encoder resolution and slip — tuning will not help

Also compare the **distance travelled according to odometry** with the **actual** distance measured with a tape. The difference between them is a calibration error, not a controller error.

Code to write

Skeleton — executing a command sequence

The whole command string arrives at once, while each motion takes seconds. There are two ways of dealing with that; **the choice and its justification are part of the assignment**.

	variant A: blocking execution	variant B: non-blocking execution
where the following commands wait	in the UART input buffer	in your own buffer inside the program

limitation	the whole string must fit in the UART buffer	none, the string may be arbitrarily long
what the program does during a motion	nothing besides control	it can read the link at the same time
stopping during a motion	impossible without extra work	possible
complexity	low	higher

To work out: the length of the string for the stage 6 route in characters, compared with the capacity of the UART input buffer in the Arduino library. The result decides whether variant A is available at all.

```
/* ===== variant A: blocking execution =====
   The loop processes character by character; a command runs to completion before
   the next character is fetched. The rest of the string waits in the UART buffer. */

void runBlocking();     /* called from loop() */

/* TODO: in the loop over characters - recognise the command letter, read the number,
         run the motion to completion, print the result line, move on to the next
         character. Note that the controller still has to run at a fixed period
         - "to completion" does not mean a function that stalls execution.        */

/* ===== variant B: non-blocking execution =====
   Reading and execution separated; commands held in your own buffer.            */

static const uint8_t QUEUE_CAP = /* TODO: decide, bearing in mind 2 kB of RAM */;

/* TODO: a type describing one pending command - the kind of motion and its argument.

   NOTE on .ino files: the Arduino environment generates prototypes for all functions
   and inserts them BEFORE the first function definition in the file. A type used in a
   signature (a struct, an enum) therefore has to be declared above that point, or the
   compiler reports "not declared in this scope" at a place where the source code
   contains no mistake at all.                                                    */

bool    queuePush(/* ... */);
bool    queuePop(/* ... */);
uint8_t queueDepth();

void execStep();   /* one step of the current motion, called at a fixed period    */

void loop() {
  handleCommands();   /* reading from the link - stage 5                          */

  /* TODO: one executor step every CTRL_DT_MS;
           fetch the next command from the buffer once the previous one has finished;
           print the result line when the motion ends.                           */
}
```

Whichever variant you choose, one rule holds: **a function that stalls execution must not appear in the control loop**. A "drive for a given time" construction built on such a function makes any reaction during the motion impossible. The manufacturer's examples are built exactly that way — one more reason for the ban at the start of the guide.

```
# Processing the three acceptance runs.
#
# Input  : for each run the deviation from B and the heading deviation
# Output : mean shift and spread; conclusion on which error component dominates
```

Checkpoint 6 — acceptance

Functionality

- ☐ Drive and turn commands executed, each ending with a result line

- ☐ The robot executes a whole command sequence sent as a single string
- ☐ A string containing unrecognised characters causes no unintended motion
- ☐ An out-of-range argument handled according to the adopted rule
- ☐ Units and the sign rule for `L` / `R` decided and documented

A → B drive

- ☐ Phase 1: route established with single commands, sequence recorded
- ☐ String length computed and compared with the input buffer capacity
- ☐ Phase 2: autonomous drive from a single string, performed three times
- ☐ Phase 3: spread of point B measured; error components separated
- ☐ Explained why the phase 2 result differs from phase 1